

Introdução à Programação orientada a objetos

Marcos Monteiro Junior

Programando com Java

Sumário

1	Introdução a Java	3
1.1	A linguagem Java	3
1.2	Máquina Virtual	3
1.3	HOTSPOT e JIT	4
1.4	Outras nomenclaturas	4
2	Referências Bibliográficas	5

1 Introdução a Java

Iniciaremos a disciplina de Orientação a Objetos abordando a linguagem Java. Escolhemos essa linguagem devido à sua versatilidade, pois ela está presente em uma ampla variedade de sistemas e dispositivos. Além disso, os conceitos de orientação a objetos aprendidos aqui podem ser aplicados em qualquer outra linguagem que suporte esse paradigma; a única diferença será a sintaxe (a forma de escrever o código), que varia conforme as regras de cada uma.

1.1 A linguagem Java

Java é uma linguagem de programação orientada a objetos, criada em um cenário onde desenvolvedores precisavam lidar manualmente com ponteiros e o complexo gerenciamento de memória (comum em linguagens como C ++), além de enfrentar o desafio de sistemas fortemente acoplados a arquiteturas de hardware e sistemas operacionais específicos. A linguagem Java surgiu para resolver esses problemas. Inicialmente projetada pela Sun Microsystems para dispositivos embarcados de baixo processamento como conversores de TV e eletrodomésticos, o objetivo central era a portabilidade sob o lema “Write Once, Run Anywhere” (Escreva uma vez, execute em qualquer lugar). Hoje, o Java ultrapassou suas origens e consolidou-se como uma das linguagens mais versáteis do mercado, sendo amplamente utilizada em diversos contextos, desde sistemas corporativos de grande escala e aplicações web até o desenvolvimento mobile.

1.2 Máquina Virtual

Em linguagens de programação como o C, o código-fonte passa por um rigoroso processo de tradução antes de rodar. Durante essa fase, entram em cena os analisadores léxicos e sintáticos, a construção de árvores sintáticas e as técnicas de recuperação de erros. O objetivo de todas essas etapas é transformar o que foi escrito em código binário (linguagem de máquina), para que possa ser executado diretamente pelo sistema operacional.

Com isso, o binário gerado do programa elaborado é executado somente para o sistema operacional que ele foi compilado. Para a utilização desse programa em outro sistema operacional todo processo de compilação deve ser executado novamente, podendo ter incompatibilidade entre bibliotecas.

Um dos grandes trunfos do Java é a portabilidade, permitindo que o mesmo código seja executado em diferentes sistemas operacionais sem a necessidade de recompilação. Isso é possível porque a linguagem utiliza uma máquina virtual *Java virtual machine* (JVM) que abstrai as particularidades de cada sistema, permitindo que o desenvolvedor foque

na lógica da aplicação sem se preocupar com as especificidades do sistema operacional (S.O.).

O código-fonte (arquivos `.java`) é compilado pelo `javac` em bytecode (arquivos `.class`). Esse bytecode é então interpretado ou compilado em tempo de execução pela JVM. Esta camada atua como um intermediário inteligente: ela pode coletar métricas de performance, gerenciar a alocação de memória e isolar a aplicação do sistema operacional hospedeiro. Esse isolamento é fundamental, pois garante que, se uma JVM for encerrada abruptamente, as demais aplicações do servidor permaneçam inalteradas, protegendo a estabilidade do ambiente como um todo (Cabral (2014)).

Logo um programa feito em Java possui cinco etapas até sua execução completa: editar, compilar, carregar, verificar e executar.

- Editar: etapa onde o programador escreve seu programa.
- Compilar: o `javac` transforma o código da etapa anterior em bytecode (arquivos `.class`).
- Carregar: Os bytecodes são carregados na memória do computador
- Verificar: verificador de bytecode examina seus bytecodes para assegurar que eles são válidos e não violam restrições de segurança do Java
- Executar: A JVM executa os bytecodes fazendo as ações programadas entrarem em funcionamento.

1.3 HOTSPOT e JIT

Na última etapa citada anteriormente, dois componentes da JVM entram em atividade, o HOTSPOT e JIT.

Para garantir um bom desempenho, a JVM utiliza a tecnologia HotSpot. A função dela é monitorar a aplicação e encontrar ‘pontos quentes’ — aquelas partes do código que rodam repetidas vezes, como o interior de um *loop*. É exatamente nessas situações que o Compilador JIT (*Just-In-Time*) é acionado, compilando sob demanda os trechos onde o desempenho poderia cair. Ao compilar em tempo de execução, o JIT consegue avaliar o código rodando e encontrar a melhor árvore sintática para otimizar aquele processo.

1.4 Outras nomenclaturas

Java Runtime Environment (JRE) é o ambiente de execução do Java pela JVM e bibliotecas, usado para executar uma aplicação Java.

Java Development Kit (JDK) é o kit de desenvolvimento Java, ele pode ser integrado a diversas IDEs de programação ou interpretadores de texto.

2 Referências Bibliográficas

Cabral, L. P. (2014). Java e orientação a objetos.